SENG 319: Software Design Patterns

Group Project 1

Detection and Refactoring of Anti-Patterns and Code Smells

Part I: Using the PTIDEJ tool analyse the CheckStyle application to detect Software Design Anti-Patterns (this includes tool specific Design smells). Answer the following questions.

1. What are the Anti-pattern types (design smells) detected by PTIDEJ? List the types and number of instances for each type.
2. In your opinion, from the results from (1), what is the Anti-pattern type (design smell) that has the most impact on the quality of the CheckStyle application? Justify your answer. [Complete answers supported with code snippets and detailed justifications will get higher marks.]
3. For the anti-pattern (design smell) instances from (1) choose **TWO** instances and refactor them. You should show the following:
    a. The reason you chose the anti-pattern (design smell) instance.
    b. The location within the CheckStyle base [code snippet(s)].
    c. The steps to refactor and solve the issue and your justification.
    d. The refactored code snippet(s) and the results from the PTIDEJ tool showing that the anti-pattern (design smell) instance has been resolved.

Part II: Using the PTIDEJ tool analyse the CheckStyle application to detect Code Smells (this includes tool specific Code smells). Answer the following questions.

1. What are the Code Smells detected by PTIDEJ? List the types and number of instances for each type.
2. In your opinion, from the results from (1), what is the Code Smell that has the most impact on the quality of the CheckStyle application? Justify your answer. [Complete answers supported with code snippets and detailed justifications will get higher marks.]
3. For the Code Smell instances from (1) choose **TWO** instances and refactor them. You should show the following:
    a. The reason you chose the code smell instance.
    b. The location within the CheckStyle base [code snippet(s)].
    c. The steps to refactor and solve the issue and your justification.
    d. The refactored code snippet(s) and the results from the PTIDEJ tool showing that the code smell instance has been resolved.

Submission:

The refactoring must be conducted on **one CheckStyle code base**. The updated CheckStyle code base must execute and work correctly. To show your work comment next to all your updates “Anti-Pattern Refactor” or “Code Smell Refactor”.

All work conducted in Part I and II should be combined within one report.

Project Submission:

a. Only ONE student should submit for each group.
b. Submit: two files:
    (1) the project report in a non-protected pdf file and
    (2) a ZIP file of the CheckStyle source code.
c. Name both files using the group's name:
    (1) *Group_Name_* Proj1_Report.pdf
    (2) *Group_Name_* Proj1_Code.zip
d. If the zip file is too large (> 20MB) upload a pdf file with a link to the zip file on your personal Google Drive.

Submission Deadline: Saturday, November 7th . 12 midnight.